NXT WAVE™

WELCOME



Authorization

Recap

Login & JWT Tokens

- Tokens
  - Access vs refresh tokens
- The Manual Way
  - What if we built login ourselves?
- Login API
  - SimpleJWT



NXT WAVE™

Agenda for Today's Session

Authorization



IDE

Initial Setup

Open IDE

Run these terminal commands step by step :

Terminal

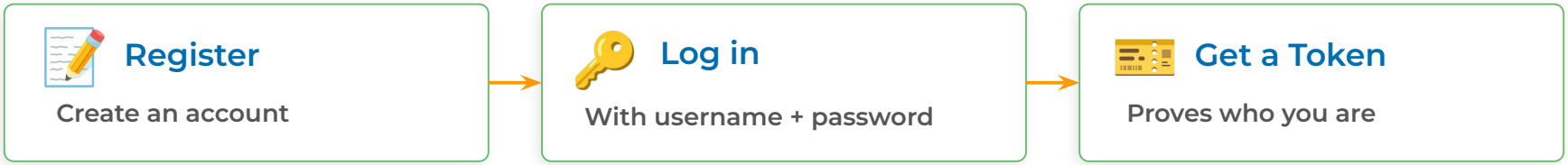
```
» python3 -m venv env
» source env/bin/activate
» pip3 install -r requirements.txt
```



Authorization

Where we Left Off

Last session, users could log in and get a token that proves who they are:



But the token doesn't do anything yet

- Right now, anyone can still add, edit or delete products even without logging in
- The door is wide open. Having a token means nothing if nothing ever checks it

Today we put locks on the doors

NXT WAVE™

Two Words – Login vs Permission

We met these last time. Now, it's all about the **second** one:

Authentication

"Who are you?"

- Logging in to prove your identity
- You log in once (authentication)

Authorization

"What are you allowed to do?"

- Logging in to prove your identity
- Then, on every action, the app checks if you're allowed (authorization)

django

Why We Need Permissions?

NXT WAVE™

Why Permissions

What ‘No Permissions’ Means?

Our product endpoints have no protection. So, right now, anyone can:

➕ Add fake products

⚠️ A random stranger can POST new products to your shop

✏️ Change any price

⚠️ Someone could edit a ₹50,000 laptop to ₹1

No real app can work like this.

We need rules about who is allowed to do what and how.

Those rules are called **permissions**

NXT WAVE™

Why Permissions

A Permission is a Checkpoint

A **permission** is a small rule DRF runs before your view to decide: allow this request or not?

01 **Request arrives**

02 **Permission checks the rule**

03 **Allowed? Run the view**

If the rule says **NO**...

- The request is blocked with an error, and your view code would never even run

Think of it as a **security guard** standing in front of each view, who checks everyone before they enter.

NXT WAVE™

django

IsAuthenticated

NXT WAVE™

Must Log In

Let’s Try It Ourselves!

Before we use DRF’s ready-made rule, let’s see what the check actually has to do.

1. Find out WHO is making the request
2. If they’re NOT logged in → block them with 401
3. If they ARE logged in → let the view run as usual

Once we’ve written this by hand, DRF shortcut will click instantly



NXT WAVE™

Authorization

request.user.is_authenticated

When a request arrives, DRF reads the token (if any) and puts a user object onto the request:

With a valid token

What's in request.user

- `request.user` → `<User: priya>`
- `request.user.is_authenticated` → `TRUE`

No token (or invalid)

What's in request.user

- `request.user` → `<AnonymousUser>`
- `request.user.is_authenticated` → `FALSE`

One field is all we require: `request.user.is_authenticated` – it's True or False

NXT WAVE™

Authorization

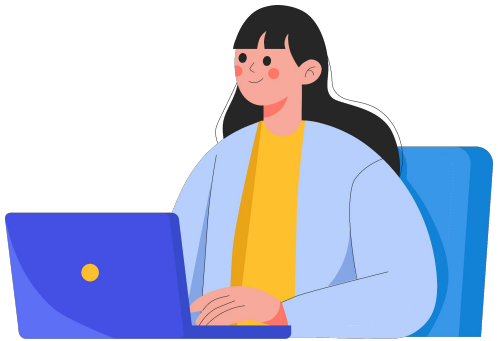
request.user.is_authenticated

For the manual approach, we use APIView – a view where we **write** the **method**:

users/views.py

Python

```
» from rest_framework.views import APIView
» from rest_framework.response import Response
»
» class MyProfileView(APIView):
»     def get(self, request):
»         # the auth check will go here
»         ...
```



NXT WAVE™

Authorization

`request.user.is_authenticated`



- APIView gives us our own `get()` — that's where the auth logic fits naturally.
- **MyProfileView** will return the logged-in user's info, and obviously it needs login to work

NXT WAVE™

Authorization

Think of an Office Building

users/views.py

Python

```python
class MyProfileView(APIView):
    def get(self, request):
        # 1) the check
        if not request.user.is_authenticated:
            return Response({"detail": "Not authenticated"}, status=401)
        # 2) return who you are
        return Response({
            "username":  request.user.username,
            "email":     request.user.email,
        })
```

No DRF magic yet; just plain Python checking a flag at the top of the method

NXT WAVE™

Authorization

Think of an Office Building

 users/views.py

Python

```
» from .views import RegisterView, MyProfileView
»
» urlpatterns = [
»     ...
»     path('profile/', MyProfileView.as_view(), name='my-profile'),
»     ...
```

Must Log In

Test it – No Token

First, hit `/users/profile/` WITHOUT logging in

</> REQUEST

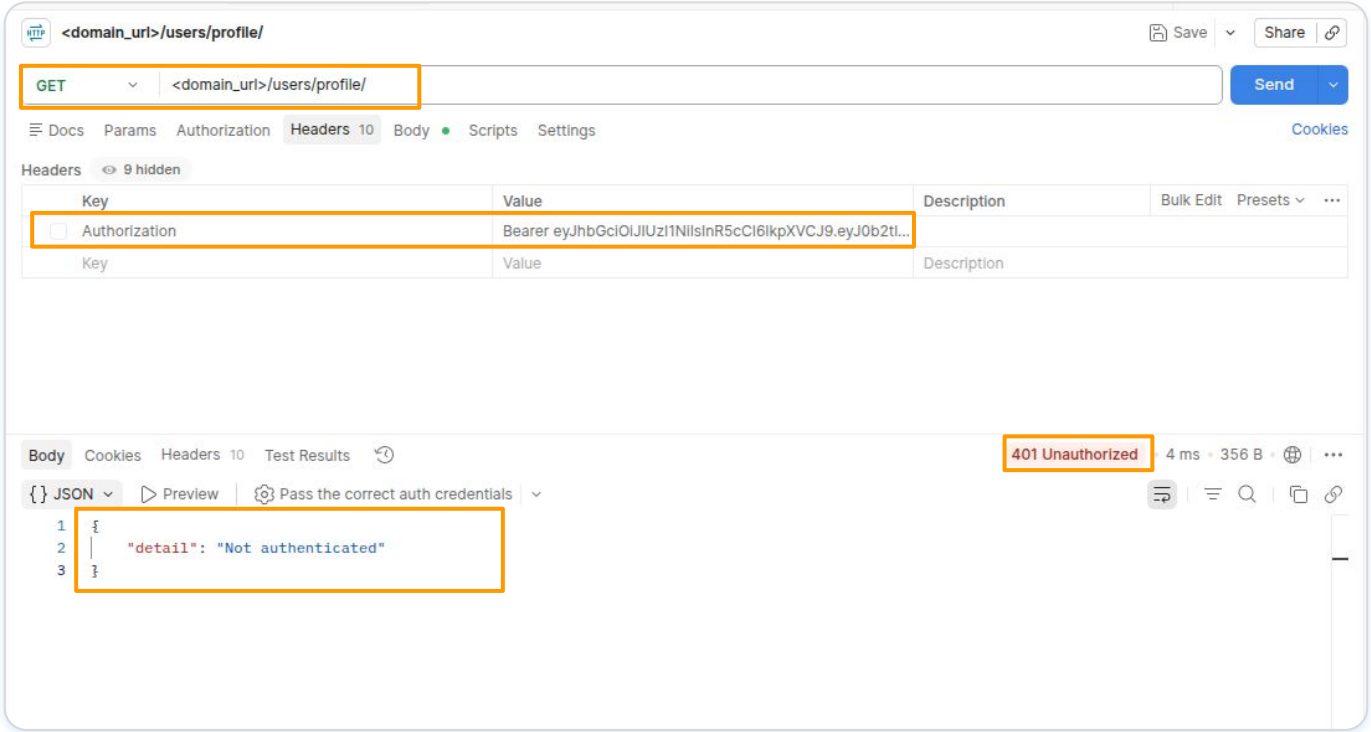
```
» GET <domain_url>/users/profile/>
```



NXT WAVE™

Must Log In

Test it – Using Postman



Must Log In

Test it – With a Valid Token

Now log in, copy the access token, and send it in the **header**:

REQUEST

```
» GET <domain_url>/users/profile/
» Authorization: Bearer eyJhbGci…your access token…
```



RESPONSE

```
HTTP 200 OK

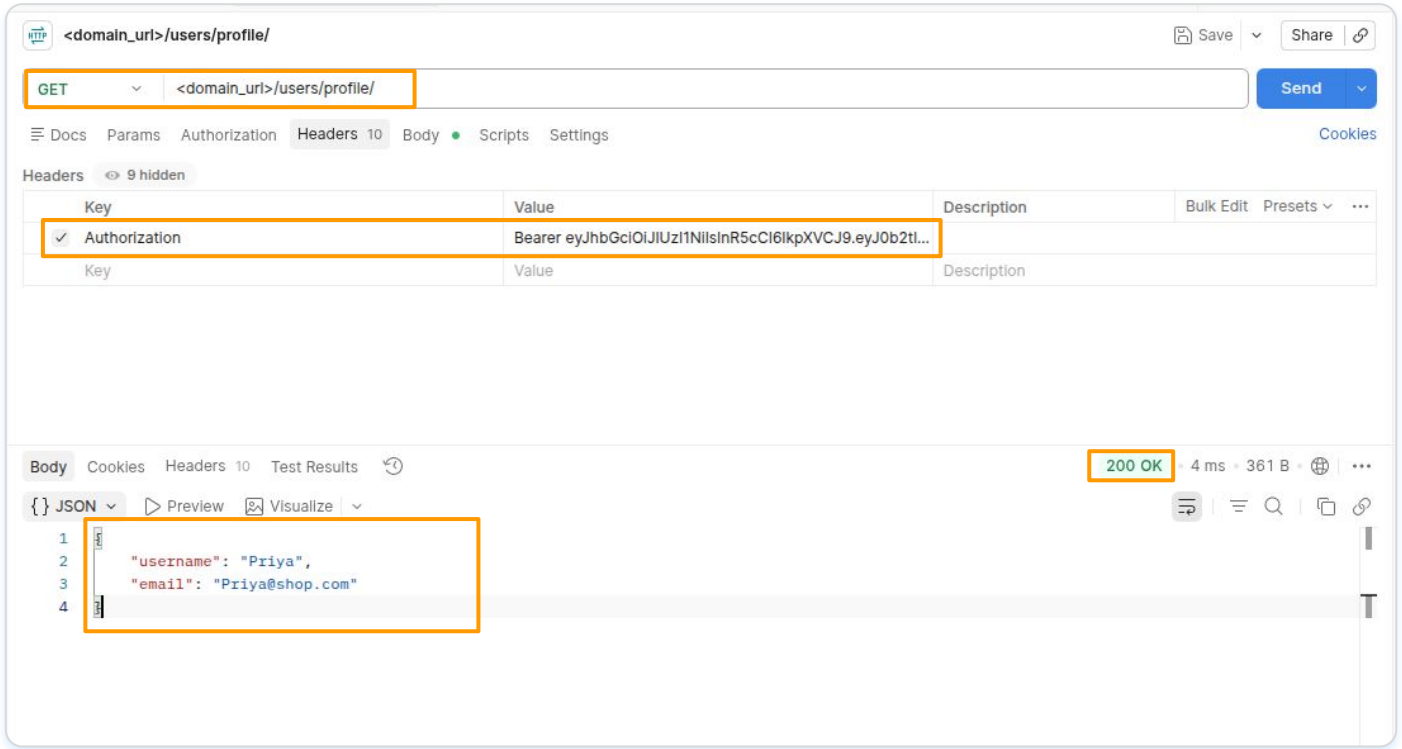
{ "username": "priya", "email": "priya@example.com" }
```

Token → `request.user` is real → check passes → profile returned!

NXT WAVE™

Must Log In

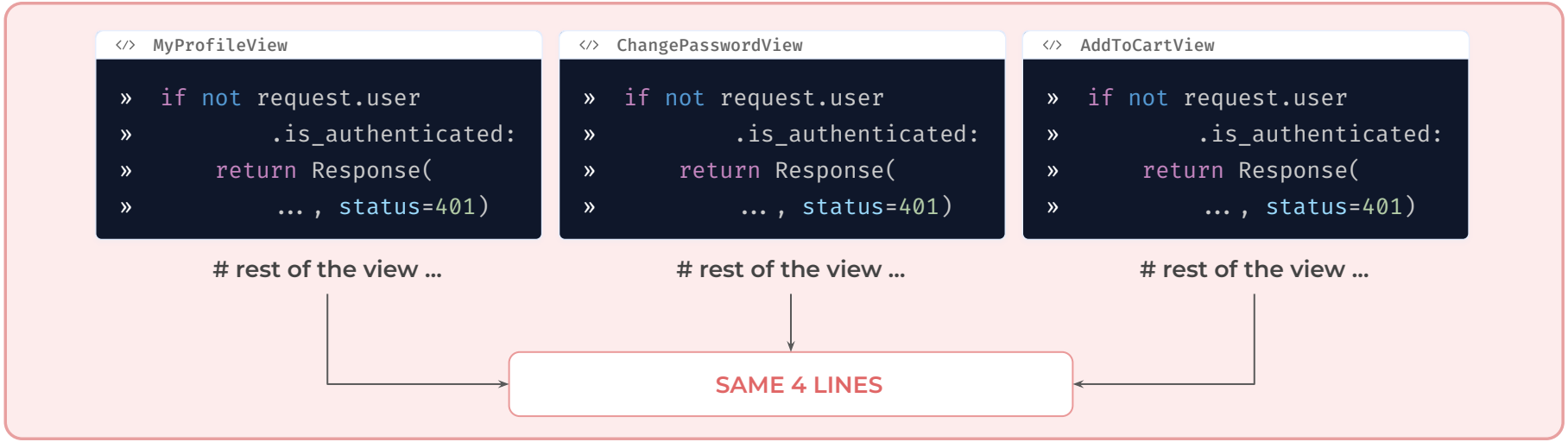
Test it – Using Postman



Must Log In

Imagine Every APIView

Lots of views need our own logic – and every one of them needs the same check at the top:



Same code, copy-pasted everywhere. Forget the check on one view → that view is wide open

Must Log In

DRF Gives Us a Shortcut

DRF's built-in rule does the exact same check – declared once, not written in every method:

Manual way *Check inside every method*

Python

```
class MyProfileView(APIView):
    def get(self, request):
        if not request.user
             .is_authenticated:
            return Response( ... , 401)
        # ... real logic ...
```

DRF way *Declare the rule once on the class*

Python

```
from rest_framework.permissions import IsAuthenticated
class MyProfileView(APIView):
    permission_classes = [IsAuthenticated]
    def get(self, request):
        # straight to real logic
        # no check needed!
```

And, the best part: this one line works on both custom APIViews and generic views like ProductListView. Let's see the syntax!

Must Log In

Add the `IsAuthenticated` Rule

Add two lines to **ProductListView** — import the rule, then list it in permission_classes:

products/views.py

Python

```
» from rest_framework.permissions import IsAuthenticated
» class ProductListView(ListCreateAPIView):
»     queryset           = Product.objects.all()
»     serializer_class   = ProductSerializer
»     permission_classes = [IsAuthenticated]   # ← the lock
```

The list of rules for this view

You must be logged in

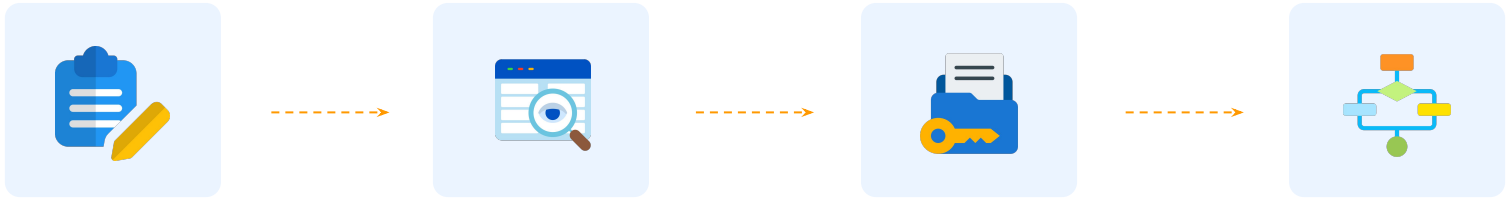
All that manual logic from earlier → one declared line. Same protection, far less code

NXT WAVE™

Must Log In

How the Token is Finally Used

Here's how `IsAuthenticated` uses the token:



The Bearer token from last session is what fills `request.user`

No token → no user → IsAuthenticated says no

NXT WAVE™

Must Log In

Two outcomes – Token or No Token

DRF's built-in rule does the exact same check - declared once, not written in every method:

No token *Status 401 Unauthorized*

JSON

```
» {
»   "detail": "Authentication
»    credentials were not
»    provided."
» }
```

Valid token *Status 200 OK*

JSON

```
» {
»   "count": 47,
»   "results": [ ... ]
» }
```

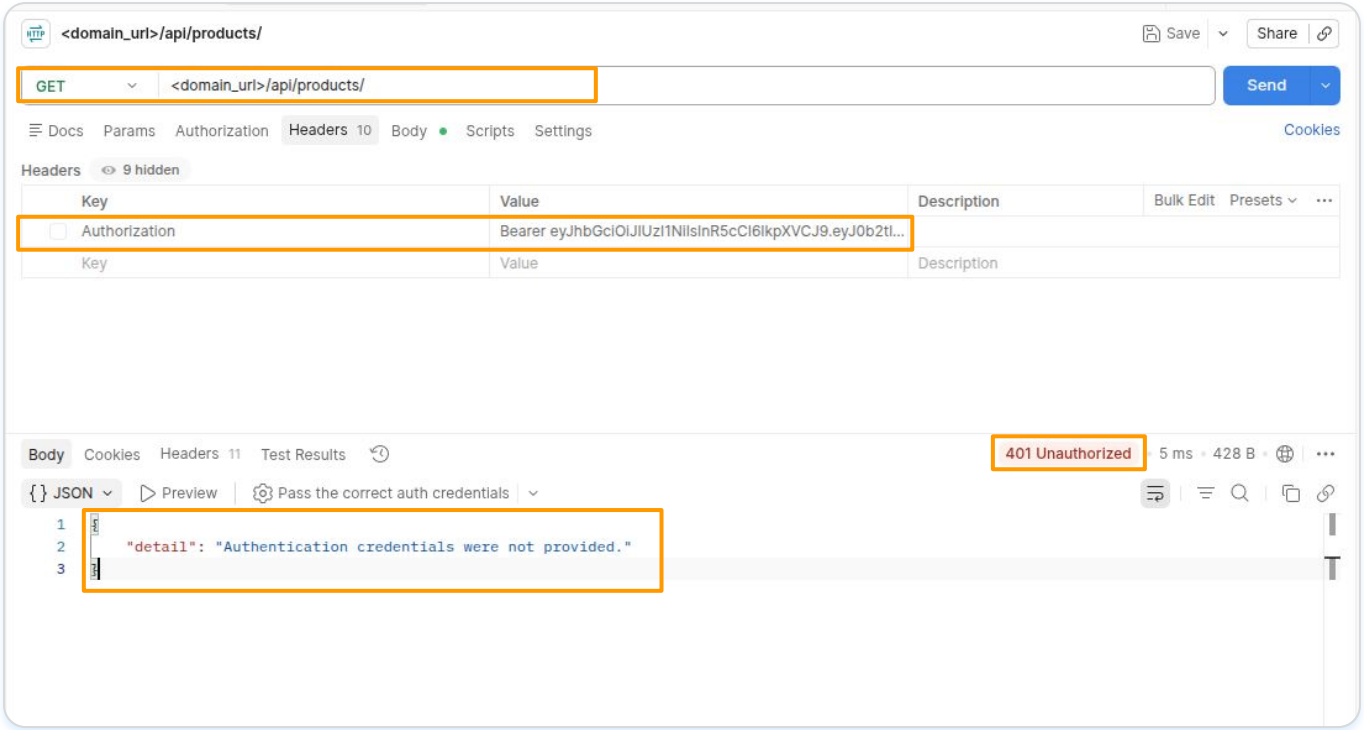
401 means 'we don't know who you are – log in first'.

The view only runs once a valid token is present

NXT WAVE™

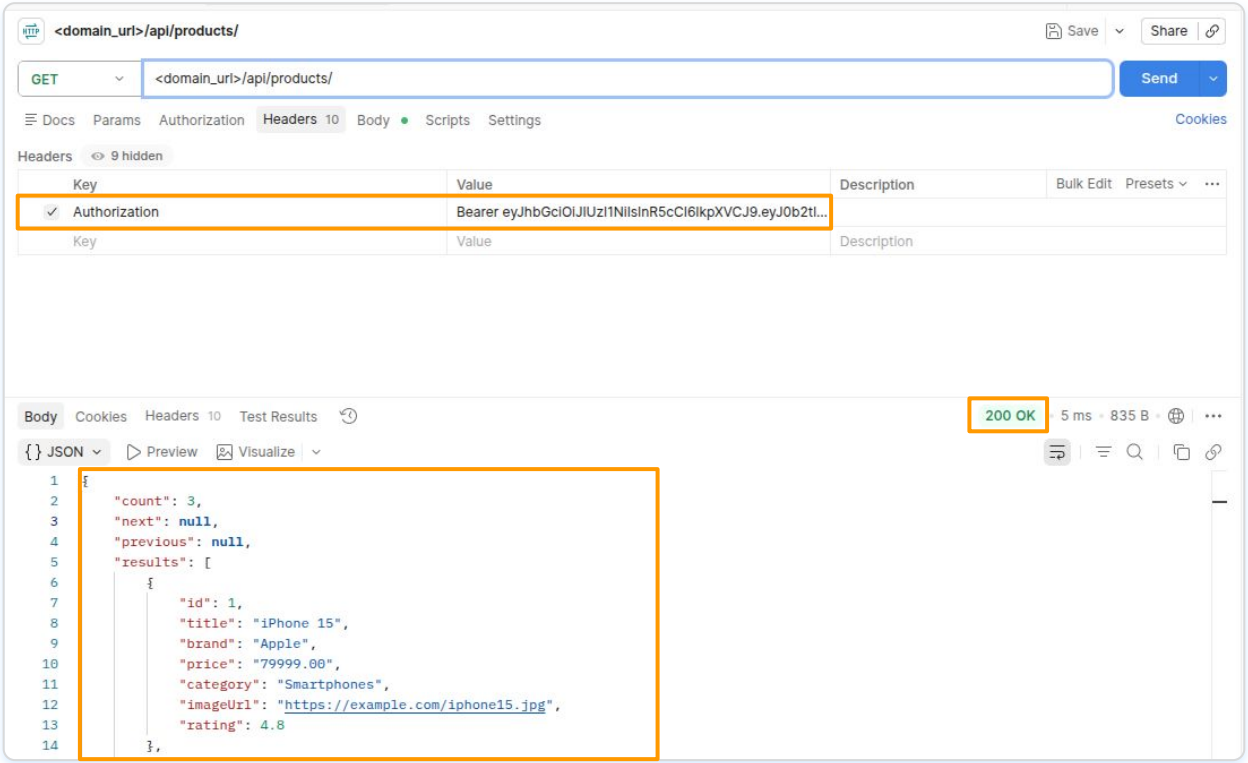
Must Log In

Testing using Postman – No Token



Must Log In

Testing using Postman – with Token



Quiz Time!



django

Read vs Write

NXT WAVE™

Read vs Write

Locked out Everyone

`IsAuthenticated` locked the whole view, but a shop needs browsing open to everyone:

Too Strict

- A new visitor can't even SEE the products unless they log in first
- That would kill an online shop — people browse before they sign up

What We Actually Want

- Anyone can view products
- But only logged-in users can ADD, EDIT or DELETE
- Open to look, locked to touch.

DRF has a ready-made rule for exactly this: `IsAuthenticatedOrReadOnly`

NXT WAVE™

Read vs Write

Reading vs Changing – ‘Safe’ Methods

DRF splits requests into two groups, based on the HTTP method used:

Unsafe – Changing

POST PUT DELETE

Creates, edits or removes data

These need a login

Safe – Just reading

GET

Just looking at data. Changes nothing

This doesn't need login

NXT WAVE™

Read vs Write

Use `IsAuthenticatedOrReadOnly`

Just swap the rule – one word change from before:



`products/views.py`

Python

```
» from rest_framework.permissions import IsAuthenticatedOrReadOnly
» class ProductListView(ListCreateAPIView):
»     queryset         = Product.objects.all()
»     serializer_class = ProductSerializer
»     permission_classes = [IsAuthenticatedOrReadOnly]
```

GET is now open to everyone; POST, PUT and DELETE require a valid token

NXT WAVE™

Read vs Write

How it Decides, Per Request

The rule looks at the method first, then decides:

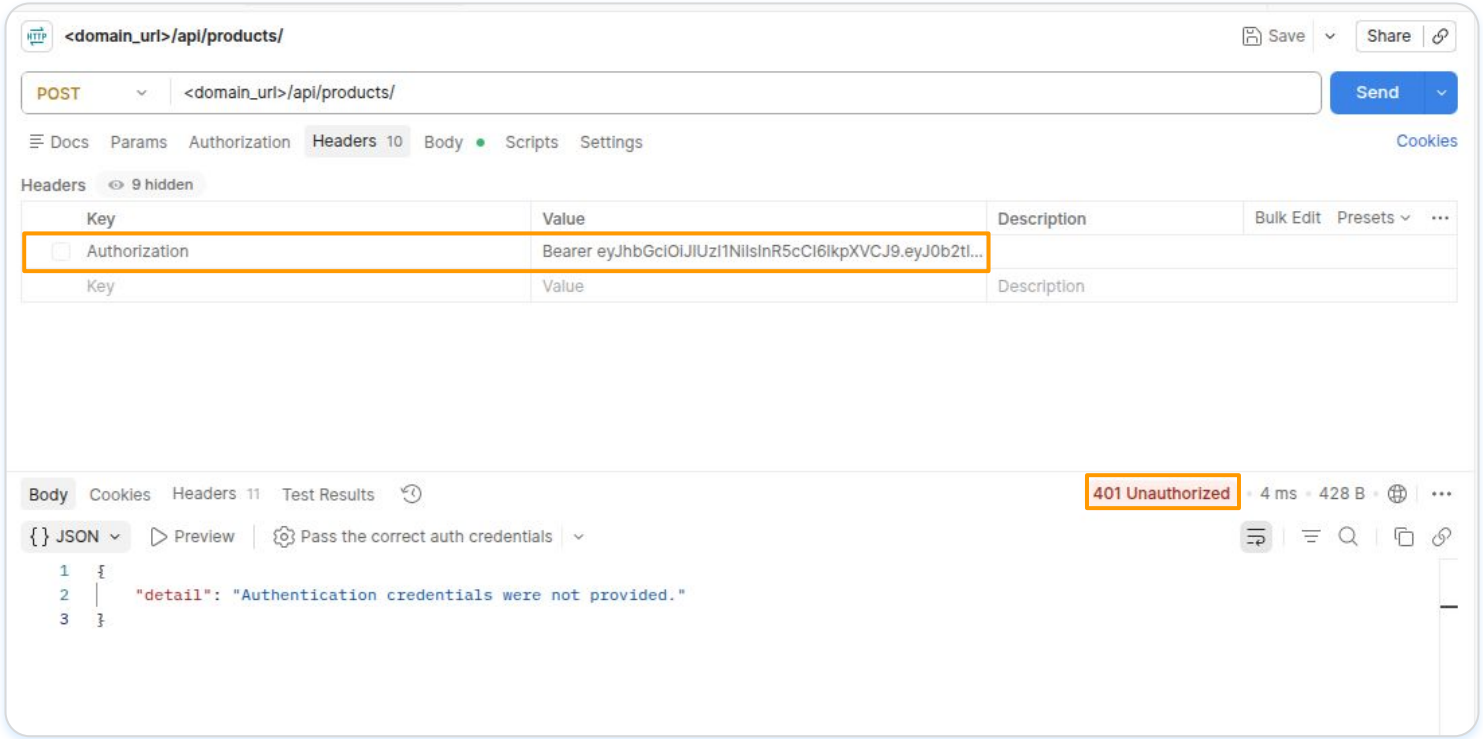


Perfect for a public shop with private editing

NXT WAVE™

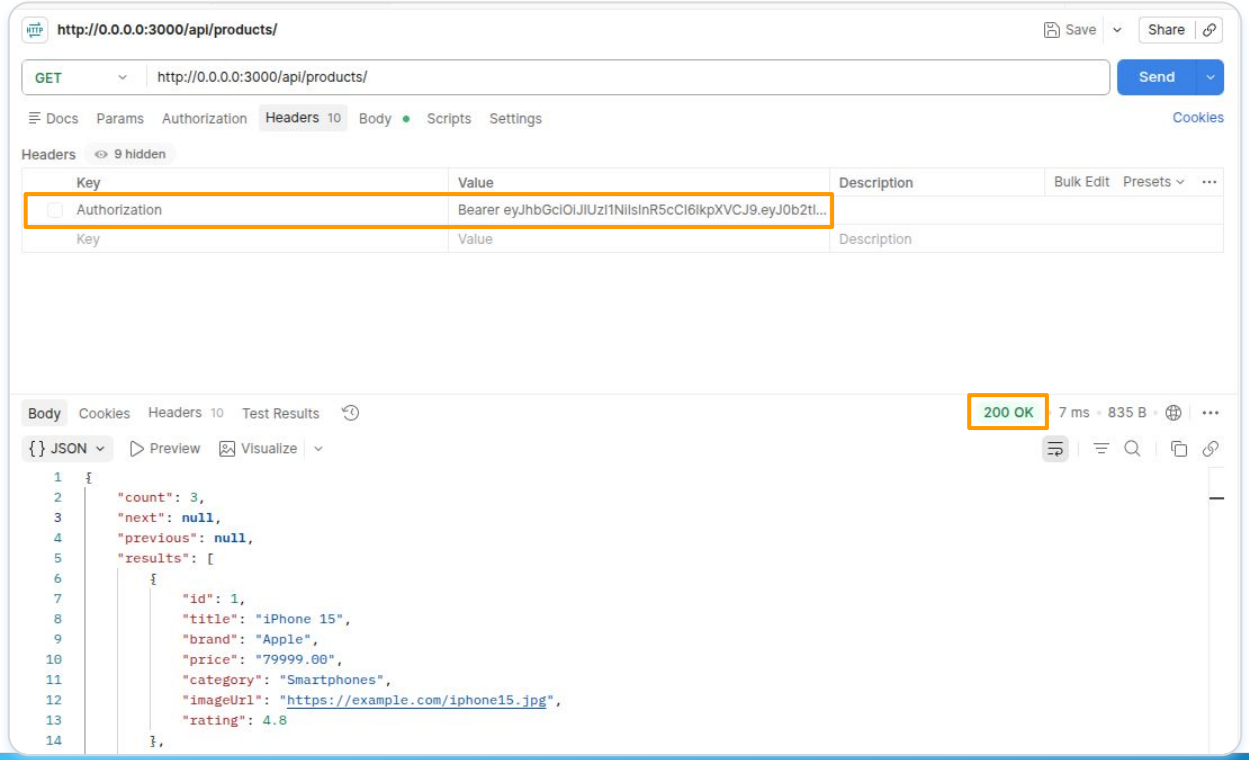
Read vs Write

Testing – (POST) No Token



Read vs Write

Testing – (GET) No Token



Read vs Write

Any Logged-in User Can Add Products

`IsAuthenticatedOrReadOnly` opens reading - but it lets ALL logged-in users write, no matter customer or seller.

📁 products/views.py

</> Python

```
» from rest_framework.permissions import IsAuthenticatedOrReadOnly
» class ProductListView(ListCreateAPIView):
»     queryset         = Product.objects.all()
»     serializer_class = ProductSerializer
»     permission_classes = [IsAuthenticatedOrReadOnly]
```

Read vs Write

Any Logged-in User Can Add Products

If we try `POST /products/` as three different users:

Anonymous

No Token

`401`

Blocked - correct

Customer (logged in)

Customer Token

`201`

Product Created - Wrong

Seller (logged in)

Seller Token

`201`

Product Created - Correct

Test It

What We Achieved

IsAuthenticated

Lock everything - must be logged in

	Read	Write
Anonymous *no token*	✗	✗
Customer *logged in*	✓	✓
Seller *user_type='seller'*	✓	✓

IsAuthenticatedOrReadOnly

Anyone reads - only logged-in users can change

	Read	Write
Anonymous *no token*	✓	✗
Customer *logged in*	✓	✓
Seller *user_type='seller'*	✓	✓

NXT WAVE™

Test It

What We Achieved



"Logged in" isn't enough. A customer should **browse** and **buy** – not add products. We need to check the ROLE. **next: a custom rule**

django

Role-based Authorization

NXT WAVE™

Role-Based

The Rule – Read vs Write

Two simple rules, based on what kind of request comes in:

WRITE

POST · PUT · DELETE

Must be a logged-in seller

- `request.user.is_authenticated`
- `AND user_type == 'seller'`

Fails either check? `403 Forbidden`

e.g. POST /products/ · PUT /products/42/ · DELETE …

READ

GET

No login needed.

- Anyone, even logged-out visitors - can browse the products

Return the data. `200 OK`

e.g. GET /products/ · GET /products/42/

NXT WAVE™

Role-Based

We Use The Role Now

'Logged in' isn't enough. A customer shouldn't be able to add products – only sellers:



Customer

Logged in

Should only browse and buy

❌ Cannot add products



Seller

Logged in

Lists items for sale

✅ Can add products.

Remember the `user_type` field we built? This is where it finally pays off

Read vs Write

First, Meet SAFE_METHODS

Before we build the rule, here's the DRF helper we'll use:

```python
»  # SAFE_METHODS is just a tuple — the HTTP methods that READ data:
»  SAFE_METHODS = ('GET', 'HEAD', 'OPTIONS')
»
»  request.method in SAFE_METHODS    # True for GET, False for POST/PUT/DELETE
```

Safe (read-only)

GET · HEAD · OPTIONS - changes nothing on the server

write

POST · PUT · PATCH · DELETE - changes data

Read vs Write

Start the Rule – `IsSellerOrReadOnly`

Create `permissions.py` in products app



products/permissions.py

python

```python
from rest_framework.permissions import BasePermission
from rest_framework.permissions import SAFE_METHODS
class IsSellerOrReadOnly(BasePermission):
```

parent class for creating your own custom permissions

NXT WAVE™

Read vs Write

Start the Rule – `IsSellerOrReadOnly`

Create `permissions.py` in products app

products/permissions.py

```python
from rest_framework.permissions import BasePermission
from rest_framework.permissions import SAFE_METHODS
class IsSellerOrReadOnly(BasePermission):
    def has_permission(self, request, view):
        # rule 1 — let reads through
        # rule 2 — block non-sellers from writing

```

parent class for creating your own custom permissions

`has_permission()` returns `TRUE` (allow) or `FALSE` (block)

Role-Based

Rule 1 – Let Reads Through

If the request is just reading, **allow it** – no other checks needed



products/permissions.py

Python

```
» class IsSellerOrReadOnly(BasePermission):
»     def has_permission(self, request, view):
»         # rule 1 — safe methods always pass
»         if request.method in SAFE_METHODS:
»             return True
»         # write methods → continue to rule 2
```

GET /products/ (anonymous)

Method in SAFE_METHODS → return True → allowed

POST /products/ (customer)

Method NOT safe → falls through to rule 2

NXT WAVE™

Role-Based

Rule 2 – Writes Need a Seller

products/permissions.py

Python

```python
class IsSellerOrReadOnly(BasePermission):
    def has_permission(self, request, view):
        # rule 1 — safe methods always pass
        if request.method in SAFE_METHODS:
            return True
        # rule 2 — must be a logged-in seller
        return (
            request.user.is_authenticated
            and request.user.user_type == 'seller'
        )
```

If we get past **rule 1**, the request is **POST/PUT/DELETE** – require an authenticated seller.

NXT WAVE™

Role-Based

Rule 2 - Writes Need a Seller



- Two rules, one class. Reading is **open**, writing is **gated** by the role.
- Drop it into `permission_classes = [IsSellerOrReadOnly]` on ProductListView — done!

Role-Based

Use It On The View

📁 products/views.py

Python

```
» from rest_framework.permissions import IsAuthenticatedOrReadOnly (Not required now)
» from .permissions import IsSellerOrReadOnly as IsSeller
» class ProductListView(ListCreateAPIView):
»     queryset         = Product.objects.all()
»     serializer_class = ProductSerializer
»     permission_classes = [IsSeller]# ← updated [IsAuthenticatedOrReadOnly] to our own rule
```

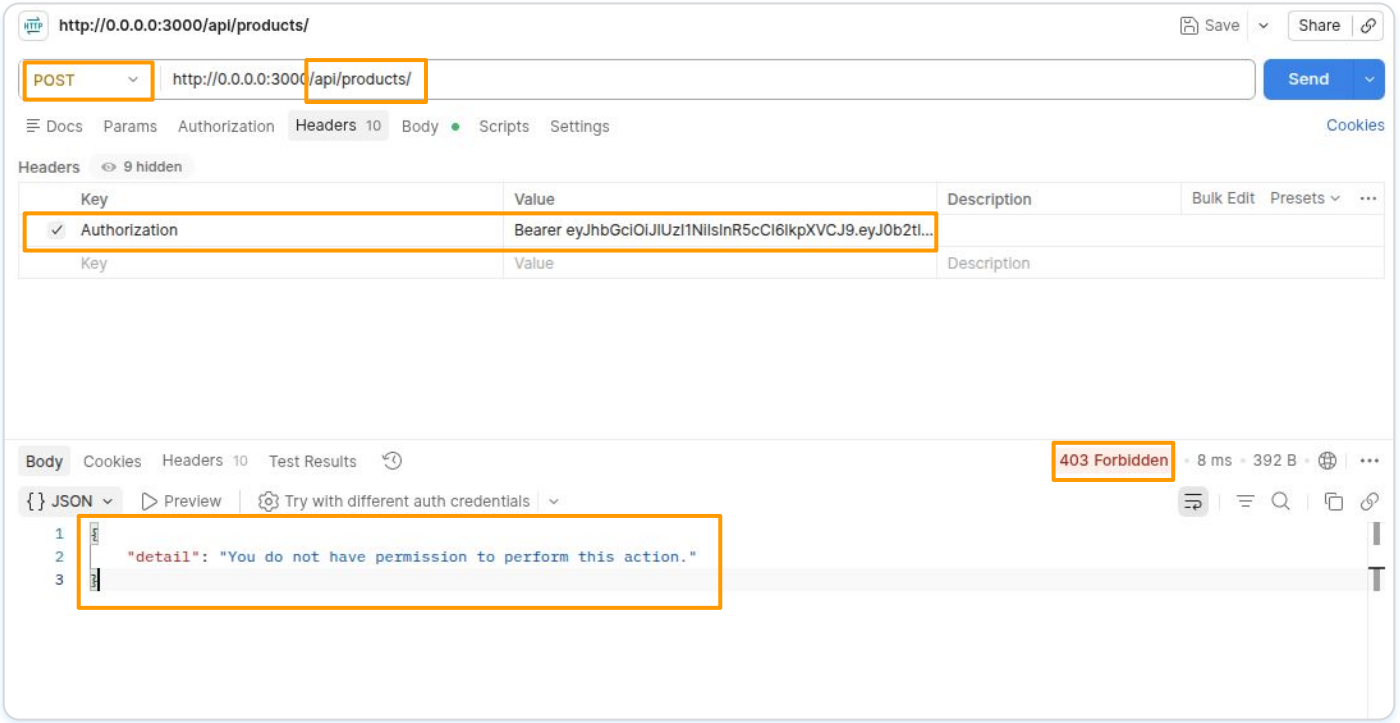
Our own rule plugs in exactly like **DRF's built-in ones**.

Now, only sellers can reach the create action.

NXT WAVE™

Role-Based

Logged in as Customer – POST



Role-Based

Use It On The View

Get can be done by anyone but **put** and **delete** by **sellers** only

products/views.py

Python

```
» from .permissions import IsSellerOrReadOnly as IsSeller
» ...
» class ProductDetailView(RetrieveUpdateDestroyAPIView):
»     queryset         = Product.objects.all()
»     serializer_class = ProductSerializer
»     permission_classes = [IsSeller]# ← [IsAuthenticatedOrReadOnly] updated to our own rule
```

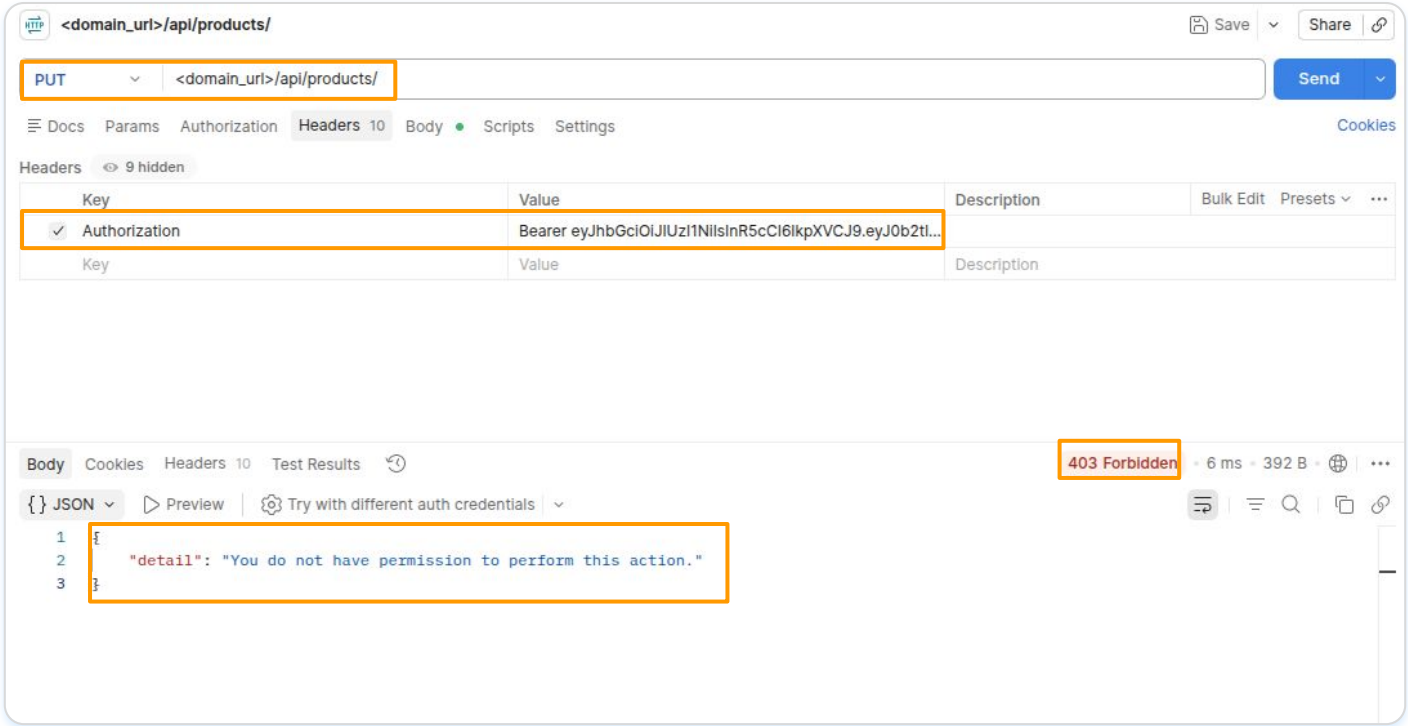
Our own rule plugs in exactly like DRF's built-in ones.

Now, only **sellers** can **reach** the **create action**

NXT WAVE™

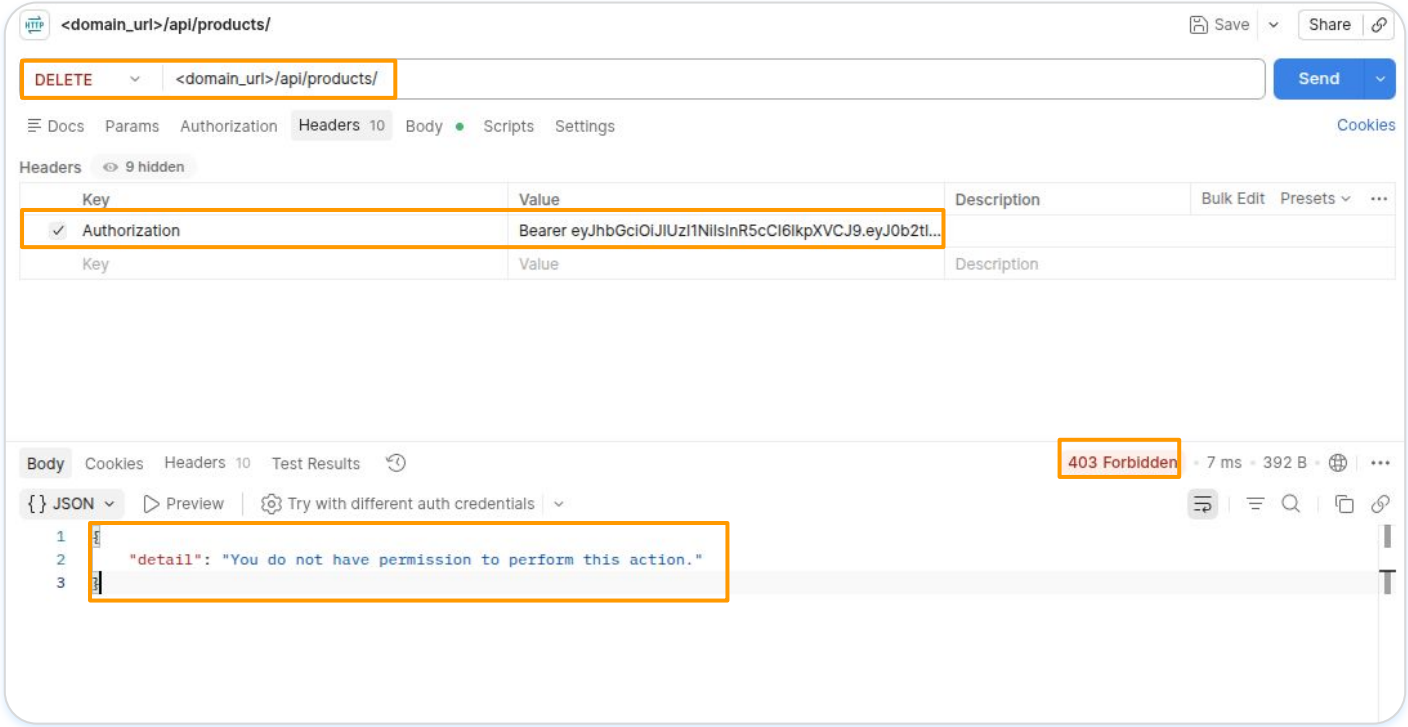
Role-Based

Logged in as Customer - PUT



Role-Based

Logged in as Customer - Delete



Role-Based

403 vs 401 – What's the Difference?

Two different 'no' answers – and they mean different things:

401 Unauthorized

Means: "I don't know who you are."

When: You sent no token, or an invalid one

Fix: Log in first

403 Forbidden

Means: "I know who you are - you're just not allowed."

When: You're logged in, but don't have the right role

Fix: Nothing — not permitted.

Role-Based

403 vs 401 – What's the Difference?



- 401 = not logged in
- 403 = logged in but not allowed
- A customer adding a product gets 403.

Role-Based

Use It On The View

Send the same `POST /api/products/` request as three different users.

Not logged in	Logged-in customer	Logged-in seller
No Token	Customer Token	Seller Token
`401` Log in first	`403` Not Allowed	`201` Product Created!

Same request, three results – that's authorization working.

The seller's role got them through

NXT WAVE™

Test It

The Three Levels of Protection

Same `permission_classes` pattern – **Three different rules**.

Pick the one that matches who should access what:

`IsAuthenticated`

Lock everything — must be logged in.

	Read	Write
Anonymous *no token*	✗	✗
Customer *logged in*	✓	✓
Seller *user_type='seller'*	✓	✓

`IsAuthenticatedOrReadOnly`

Anyone reads — only logged-in users can change DRF built-in — when role doesn't matter.

	Read	Write
Anonymous *no token*	✓	✗
Customer *logged in*	✓	✓
Seller *user_type='seller'*	✓	✓

`IsSeller` (custom)

Anyone reads — only sellers can write Used on ProductListView

	Read	Write
Anonymous *no token*	✓	✗
Customer *logged in*	✓	✗
Seller *user_type='seller'*	✓	✓

Same plug-in pattern — only the rule changes.

NXT WAVE™

Quiz Time!

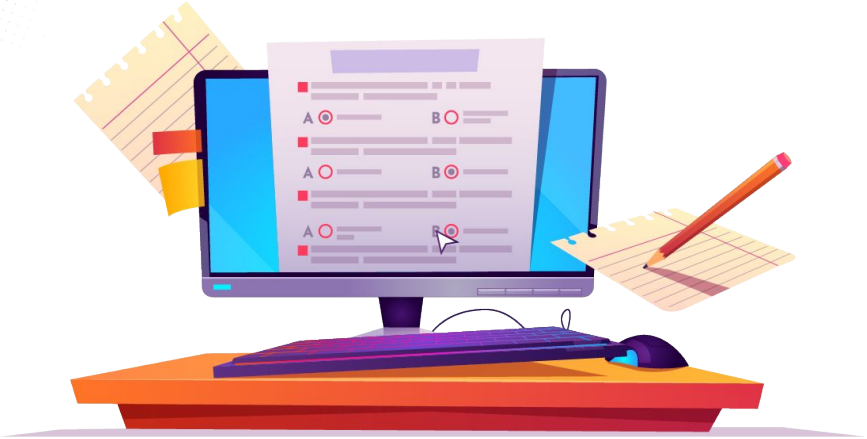


Key Takeaways

- Why Permissions?
- `IsAuthenticated`
- Read vs Write
- Role-based Authorization



Practice



Next Session

Prime Products & Seller Ownership

NXT WAVE™

THANK YOU

NXT WAVE™

ALL THE BEST